

WEEK-6 Implement deep learning Techniques for image segmentation

Install Required Tools

```
pip install tensorflow opencv-python
```

#Library Imports

```
import cv2
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras.applications import MobileNetV2
```

```
from tensorflow.keras import layers, models
```

Load the input image

```
image_path = r'C:\Exp\Alpha.JPG'
```

```
input_image = cv2.imread(image_path)
```

```
input_image = cv2.resize(input_image, (128, 128)) # Resize for UNet model
```

```
input_image = input_image / 255.0 # Normalize
```

UNet Model Definition (You can also use a pre-trained model)

```
def unet_model(input_shape=(128, 128, 3)):
```

```
    inputs = layers.Input(shape=input_shape)
```

Encoder: Downsampling

```
conv1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(inputs)
```

```
conv1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(conv1)
```

```
pool1 = layers.MaxPooling2D(pool_size=(2, 2))(conv1)
```

```
conv2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(pool1)
```

```
conv2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(conv2)
```

```
pool2 = layers.MaxPooling2D(pool_size=(2, 2))(conv2)
```

Bottleneck

```

conv3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(pool2)
conv3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(conv3)
# Decoder: Upsampling
up1 = layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(conv3)
up1 = layers.concatenate([up1, conv2], axis=-1)
conv4 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(up1)
conv4 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(conv4)

up2 = layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(conv4)
up2 = layers.concatenate([up2, conv1], axis=-1)
conv5 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(up2)
conv5 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(conv5)
outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(conv5) # Sigmoid for
binary segmentation

model = models.Model(inputs=[inputs], outputs=[outputs])
return model

```

Initialize and compile the UNet model

```

model = unet_model()

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Assuming model has been trained or using a pre-trained one
# If you have a pre-trained model, you can load it using
# model.load_weights('path_to_weights.h5')

```

Preprocess the image for prediction

```

input_image_batch = np.expand_dims(input_image, axis=0)

```

Predict segmentation

```
segmentation_output = model.predict(input_image_batch)
```

Post-process the segmentation output

```
segmentation_output = (segmentation_output > 0.5).astype(np.uint8)
```

```
segmentation_output = np.squeeze(segmentation_output) # Remove single batch dimension
```

Display original image and segmentation output

```
cv2.imshow('Original Image', input_image)
```

```
cv2.imshow('Segmentation Output', segmentation_output * 255) # Scale to viewable range
```

Save the segmented image

```
cv2.imwrite(r'C:\Exp\segmented_output.JPG', segmentation_output * 255)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```